9 Async gözləyərkən threadi boş saxlamağa kömək edir

Kafe tətbiqi uzaq ödəniş xidmətindən cavab gözləyir. Cavab gələnə qədər proqramın həmin yerdə hesablayacağı iş yoxdur. Sadəcə başqa sistemin işini bitirməsi lazımdır.

İki yanaşma ola bilər. Kod icra edən axını gözləmədə tuturuq. Yaxud cavab gələndə davam etmək üçün məlumatı saxlayıb axını başqa işə buraxırıq.

Thread prosesdə kod icra edən axındır. **Asinxron gözləmə** müəyyən əməliyyatın tamamlanmasını gözləyərkən həmin thread-i bloklamamağa imkan verir.

Gözləmə fərqi

```
Bloklayan gozleme:
    Thread bu yerde gozleyir, basqa is gormur.

Asinxron gozleme:
    Davamin haradan olacagi saxlanir.
    Cavab hazir olanda davam kodu isleyir.
```

Bu, uzaq xidmətin öz işini avtomatik sürətləndirmir. Cavab yenə vaxtında gəlir. Qazanc odur ki, gözləmə zamanı thread başqa iş üçün əlçatan qala bilər.

Bir əməliyyatın cavab müddəti ilə serverin eyni müddətdə neçə işə xidmət etməsi ayrı göstəricilərdir. Birincisinə latency, ikincisinə throughput deyilir.

Task işi təmsil edir amma threadin özü deyil

Ödəniş başladılır. Dərhal yekun cavab yoxdur, amma onun tamamlanmasını təmsil edən obyekt alırıq. .NET-də belə tamamlanma üçün **Task** istifadə olunur.

`Task` nəticə dəyəri olmayan tamamlanmanı, `Task<int>` isə tamamlananda tam ədəd verən işi təmsil edə bilər. `<int>` nəticənin tipini göstərir.

Task barədə soruşduğumuz məlumat

```
Is bitibmi?
Ugurlu bitibse netice nedir?
Xeta ile bitibmi?
Legv olunubmu?
```

Task-ın olması bütün gözləmə müddətində ayrıca thread tutulması demək deyil. Şəbəkə cavabını əməliyyat sistemi izləyib hazır olanda xəbər verə bilər.

ThreadPool .NET-in qısa işlər üçün idarə etdiyi işçi thread-lər mexanizmidir. `Task.Run` işi adətən həmin pool-da planlayır. Amma bütün Task-lar Task.Run işi deyil. Artıq tamamlanmış Task və xarici cavabı gözləyən Task da mümkündür.

Yadda saxla: Task “bu əməliyyatın tamamlanmasını və nəticəsini izləyə bilərəm” deməkdir. “Bu əməliyyatın həmişə ayrıca bir thread-i var” demək deyil.

Async və await işləyəndə kod hansı sırada gedir

Bu kiçik C# nümunəsini addım-addım oxuyaq:

C#

```
static async Task DemoAsync()
{
    Console.WriteLine("A");
    await Task.Delay(1000);
    Console.WriteLine("B");
}

Task work = DemoAsync();
Console.WriteLine("C");
await work;
```

Əvvəl DemoAsync çağırılır və **A** yazılır. `Task.Delay(1000)` təxminən 1000 millisaniyə sonra tamamlanan gözləmə yaradır. Bir saniyə 1000 millisaniyədir.

Delay hələ bitmədiyi üçün `await` yerində metodun davamı saxlanır və çağıran kod davam edə bilir. Ona görə **C** yazılır. Delay tamamlananda metod qaldığı yerdən davam edir və **B** yazılır. Gözlənən sıra **A, C, B**-dir.

`async` metodda belə gözləmələr yazmağa imkan verir. Metod çağırılan kimi ilk tamamlanmamış await-ə qədər adi qaydada işləyir. **Async yazmaq kodu əvvəldən avtomatik başqa thread-ə keçirmir.**

Əgər await olunan Task artıq bitibsə metod dayanmadan davam edə bilər. Hər await mütləq başqa thread-ə keçid demək deyil.

Davam harada saxlanır və I O ilə CPU niyə fərqlidir

Metod gözləyəndə hara qayıdacağını və lazım olan dəyərləri saxlamalıdır. Kompilyator, C# mətnini işləyən formaya çevirən alət, bunun üçün **state machine** yaradır. Bu, metodun hansı mərhələdə olduğunu saxlayan mexanizmdir. Gözləmədən sonrakı kod **continuation**, davam kodu adlanır.

İndi işin əsas vaxtının hara getdiyini ayıraq:

İşin növü	Əsas vaxt nəyə gedir
I/O-bound	Şəbəkə, disk və ya baza cavabını gözləməyə.
CPU-bound	Prosessorda hesablama aparmağa.

I/O məlumatın proqramdan xaricə çıxması və ya xaricdən gəlməsidir. **CPU** hesablamaları yerinə yetirən prosessordur.

Şəkli serverdən almaq I/O üstünlüklü, onun üzərində ağır hesablama etmək CPU üstünlüklü ola bilər.

C# istifadə fraqmentləri

```
// Hazir asinxron I/O emeliyyatini gozle:
string text = await http.GetStringAsync(url);

// Lazim olanda CPU isini pool-da planla:
int value = await Task.Run(() => Calculate());
```

`http` .NET-dən HTTP sorğuları göndərən HttpClient obyektidir. `url` ünvan, `Calculate` hesablayan metodudur. Sinxron baza çağırışını Task.Run ilə sarımaq onu həqiqi asinxron baza metoduna çevirmir. Serverdə əlavə Task.Run ümumi hesablama gücünü artırmır.

WhenAll müstəqil gözləmələri üst üstə salır

Qiymət sorğusu 300, ehtiyat sorğusu 500 millisaniyə gözləyir. Biri o birinin nəticəsindən asılı deyil.

C# ardıcıl variant

```
var price = await GetPriceAsync();
var stock = await GetStockAsync();
```

İkinci iş birinci bitəndən sonra başlayır. Sadə modeldə gözləmə `300 + 500 = 800` millisaniyədir.

İndi əvvəl hər ikisini başladaq:

C# müstəqil işlər üçün variant

```
var priceTask = GetPriceAsync();
var stockTask = GetStockAsync();

await Task.WhenAll(priceTask, stockTask);

var price = await priceTask;
var stock = await stockTask;
```

`GetPriceAsync` qiyməti, `GetStockAsync` ehtiyatı əldə edən nümunə metod adlarıdır. Onların əvvəl çağırılması işləri başladır. **WhenAll** verilmiş bütün Task-lar tamamlananda tamamlanan Task verir. Onları avtomatik ayrıca CPU nüvələrində işlətmək vədi vermir.

İdeal sadə gözləmə modelində ümumi müddət təxminən ən uzun gözləməyə, 500 millisaniyəyə yaxın ola bilər. Həqiqi vaxt xarici xidmətlərdən və limitlərdən asılıdır.

Şərt: işlər müstəqil olmalıdır. Eyni DbContext-də iki paralel baza əməliyyatını bu cür işlətmək olmaz. Eyni context eyni anda bir neçə əməliyyata uyğun deyil.

Concurrency ilə parallelism fərqi

Concurrency bir neçə işin vaxt baxımından üst-üstə irəliləməsidir. Qiymət cavabını gözləyərkən ehtiyat sorğusunu başlatmaq bunun nümunəsidir. Hər an bir thread-də kod işləməsi də mümkün ola bilər.

Parallelism iş hissələrinin həqiqətən eyni anda müxtəlif icra resurslarında hesablanmasıdır. Bir neçə prosessor nüvəsində müxtəlif şəkillərin emalı nümunədir.

İki müxtəlif fikir

```
Concurrency:
    Iki xarici cavabi eyni vaxt araliginda gozleyirik.

Parallelism:
    Iki hesablamani heqiqeten eyni anda icra edirik.
```

Async əsasən gözləmə ilə işləmə üsuludur. Paralellik isə eyni anda hesablama aparmağa aiddir. Onlar birlikdə istifadə edilə bilər, amma bir-birinin adı deyil.

Hər işi limitsiz eyni anda başlatmaq da düzgün deyil. 50 min sorğunu birdən xarici xidmətə göndərmək həmin xidməti və bağlantı ehtiyatını tükədə bilər.

Buna görə bəzi işlərdə maksimum paralel əməliyyat sayı seçilir. Məsələn, eyni anda yalnız 4 fayl endirilsin. Dörd rəqəmi universal düzgün rəqəm deyil; xidmətin limiti və resurslar üzrə seçilir.

Yadda saxla: “bir neçə iş başladı” deyəndə onların gözləmələri üst-üstə düşür, yoxsa hesablamaları həqiqətən paralel işləyir, bunu dəqiqləşdir.

Xəta olanda WhenAll və WhenAny nə edir

Bir Task xəta ilə bitə bilər. Xəta obyektinə **exception** deyilir. Onu await edəndə xəta çağıran koda ötürülə bilər.

C#

```
try
{
    await Task.WhenAll(priceTask, stockTask);
}
catch (Exception error)
{
    Console.WriteLine(error.Message);
}
```

`try` yoxlanılan iş blokudur. `catch` uyğun xətanı tutur. `error.Message` xətanın mesajıdır.

WhenAll işlərin hamısının tamamlanmasını gözləyir. Biri xəta verəndə qalanları avtomatik dayandırmır. Ümumi Task bütün xətaları öz Exception məlumatında saxlaya bilər; await isə adətən həmin xətalardan birini yenidən atır. Hamısı lazımdırsa ümumi Task-ın xəta toplusuna baxılır.

WhenAny ilk tamamlanan Task-ı verir. Tamamlanma uğur, xəta və ya ləğv ola bilər. Verilən Task-ı ayrıca await etmək onun həqiqi nəticəsini və ya xətasını almaq üçündür.

WhenAny də qalan işləri avtomatik ləğv etmir. “Birincisi hazırdır” ilə “o biriləri dayandırıldı” ayrı hadisələrdir.

Əsas nəticə: tamamlanmanı izləmək, xətanı almaq və digər işləri ləğv etmək ayrı qərarlardır.

CancellationToken işi zorla öldürmür

Müştəri gözləməkdən vaz keçir. Və ya iş üçün ayrılmış 5 saniyə bitir. Davam etmək istəmədiyimizi əməliyyata bildirmək lazımdır.

CancellationToken ləğv istəyi barədə siqnalı daşıyır. **CancellationTokenSource** həmin siqnalı yaradan və ləğv edən obyektdir.

C#

```
using var stop = new CancellationTokenSource();
stop.CancelAfter(TimeSpan.FromSeconds(5));

string text = await http.GetStringAsync(url, stop.Token);
```

`CancelAfter` müddət bitəndə ləğv istəyir. `TimeSpan` vaxt müddətini bildirir. Token http metoduna ötürülür ki, həmin əməliyyat siqnalı nəzərə ala bilsin. `using` iş bitəndə source-un resurslarını təmizləyir.

Öz uzun dövrümüzdə də siqnalı vaxtaşırı yoxlaya bilərik. `ThrowIfCancellationRequested()` istək varsa ləğv xətası verir. Buna əməkdaşlığa əsaslanan ləğv deyilir: kod siqnalı qəbul edir və dayanır.

Timeout müəyyən vaxtdan artıq gözləməməkdir. Gözləmənin bitməsi işin özünün mütləq dayandığını göstərmir. Əməliyyatın dayanması lazımdırsa onun cancellation dəstəyi və token-in ona ötürülməsi əhəmiyyətlidir.

İstifadəçi ləğvi ilə vaxt sərhədini birləşdirən linked token də yaratmaq olar: hər iki mənbədən biri ləğv istəyəndə ümumi siqnal verilir.

Result və Wait niyə problem yarada bilər

`.Result` və `.Wait()` Task bitənə qədər çağıran thread-i bloklaya bilər. UI proqramında bu ekranı dondura bilər. **UI** istifadəçi interfeysidir; onun xüsusi thread-i düymə və ekran yeniləmələrini idarə edir.

Dairəvi gözləmə nümunəsi

```
UI thread: ".Result ile metodun bitmesini gozleyirem."

Metodun davami: "Bitmek ucun UI thread-de islemeliyem."

UI neticeni, netice ise UI thread-i gozleyir.
```

Belə irəliləyə bilməyən qarşılıqlı gözləmə **deadlock** adlanır.

SynchronizationContext await-dən sonra davamın hansı mühitdə işləməsini bildirə bilən mexanizmdir. UI-də bu çox vaxt UI thread-idir. `ConfigureAwait(false)` həmin await üçün bu mühitə qayıtmaq tələbini azaltmağa imkan verir. “Mütləq yeni thread aç” və ya “bütün deadlock-ları düzəlt” demək deyil.

ASP.NET Core-da standart klassik request SynchronizationContext yoxdur. Amma bloklama yenə çoxlu thread-i gözləmədə tutub başqa işlər üçün thread çatışmazlığı yarada bilər. Buna **ThreadPool starvation** deyilir.

İlkin seçim: asinxron işi mümkün olduqca çağırış zənciri boyunca await et. Gözləmə problemini `.Result` ilə yuxarıya gizlətmə.

Async void və nəzarətsiz fon işi

`async Task` metodu çağırana tamamlanmanı izləmək üçün Task verir. `async void` isə belə Task vermir. Ona görə onu adi metod kimi await edib tamamlanmasını və xətasını izləmək olmur.

Əsas istisna void imzası tələb edən hadisə işləyicisidir. Məsələn, UI düyməsinin click handler-i. Handler hadisə gələndə çağırılan koddur. Adi iş metodları üçün Task və ya Task nəticəsi olan variant seçilir.

Fire and forget işi başladıb sonunu izləməməkdir. HTTP sorğusunda işi belə başlatmaq onun etibarlı bitməsinə zəmanət vermir.

Ömür problemi

```
Sorqu baslayir, DbContext alir.
    |
    +-- gozlenilmeyen fon isi hemin context-i tutur.

Sorqu bitir, scope temizlenir.
Fon isi hele context-den istifade etmek isteyir.
```

DI dərsində scope-un xidmətləri hansı sərhədə qədər saxladığını gördük. Fon işi sorğudan uzun yaşayırsa onun resursları da ayrıca idarə olunmalıdır.

Etibarlı fon emalı üçün tətbiqin idarə etdiyi worker, öz scope-u və iş itməməlidirsə davamlı növbə istifadə oluna bilər. **Worker** arxa planda işi emal edən hissədir. Davamlı növbə isə proqram bağlansa da işi saxlayan mexanizmdir.

Resurs await bitənə qədər açıq qalmalıdır

Fayldan mətn oxuyuruq. Reader obyektini oxuma tamamlanmamış bağlasaq başladılmış iş artıq bağlanmış resursla qarşılaşa bilər.

C#

```
static async Task<string> ReadAsync(string path)
{
    using var reader = new StreamReader(path);
    return await reader.ReadToEndAsync();
}
```

`StreamReader` fayldan mətn oxuyan obyektdir. `path` faylın yoludur. `ReadToEndAsync` sonadək oxumağı təmsil edən Task verir. Await onun bitməsini gözləyir, sonra metoddan çıxırıq və using reader-i təmizləyir.

Burada await-i düşünmədən çıxarıb hələ tamamlanmamış Task-ı qaytarmaq resursun tez bağlanmasına səbəb ola bilər. **Kodun qısalması həmişə eyni davranışı qorumaq demək deyil.**

İndi **ValueTask** adını da yerinə qoyaq. O, bəzi hallarda hazır nəticəni ayrıca Task yaratmadan təmsil edə bilər. Çox tez-tez dərhal tamamlanan və ölçülmüş xərcin vacib olduğu əməliyyatlarda faydalı ola bilər.

ValueTask həmişə stack-də saxlanır demək deyil. **Stack** çağırışlara aid məlumatın saxlandığı yaddaş sahəsidir. ValueTask başqa obyektin sahəsində də yerləşə bilər; yer onun harada istifadə edilməsindən asılıdır. Ümumi qayda kimi onu bir dəfə await et. Dəfələrlə gözləmək lazımdırsa Task müqaviləsi daha rahatdır. Sadə tətbiq metodlarında başlanğıc seçim çox vaxt Task-dır.

Eyni məlumatı paralel dəyişəndə nəticə itə bilər

Sayğac 0-dır. İki thread `counter++` edir. Hər iki artım saxlanarsa nəticə 2 olmalıdır. Amma ++ oxu, artır, yaz addımlarından ibarətdir:

Addım	Thread A	Thread B
Oxuma	0 oxuyur	0 oxuyur
Hesablama	1 hesablayır	1 hesablayır
Yazma	1 yazır	1 yazır

Son nəticə 1 qala bilər. Nəticənin nəzarətsiz icra sırasından asılı olmasına **race condition** deyilir.

`Interlocked.Increment(ref counter)` artımı digər paralel artımlara qarşı bölünməz əməliyyat edir. **Atomic** burada həmin addımın araya qarışdırılmadan yerinə yetirilməsi deməkdir. `ref` dəyişənin özünə istinad ötürülməsini bildirir.

Lock müəyyən kod hissəsinə eyni anda girişi məhdudlaşdırır. Adi lock bloku daxilində await istifadə olunmur. Asinxron gözləmə ilə giriş limiti üçün `SemaphoreSlim.WaitAsync` uyğun ola bilər. Semaphore müəyyən sayda icazə saxlayır; iş bitəndə icazə geri verilir.

Nəticə: async yazmaq paylaşılan məlumatı avtomatik təhlükəsiz etmir. Bir işi gözləmək ilə eyni məlumata paralel girişi idarə etmək ayrı məsələlərdir.

Bütün dərsin məntiqini bir işə tətbiq et

Kafe ekranı qiymət və ehtiyat istəyir. Bu iki xarici sorğu bir-birindən asılı deyilsə əvvəl ikisini başlatmaq, sonra WhenAll ilə nəticələri gözləmək olar. Eyni DbContext-i paralel paylaşmaq isə olmaz.

İstifadəçi ekranı bağlayanda ləğv token-i əməliyyatlara ötürülür. Token dayanma istəyidir; hər işi zorla öldürən mexanizm deyil.

Nəticə alınanda UI görünüşü yenilənir. UI tələb olunan kod həmin mühitin thread qaydasına əməl etməlidir. Eyni thread-ə həmişə qayıdacağına və ya hər await-də thread-in dəyişəcəyinə dair ümumi hökm yoxdur.

Özünü yoxla

Task ayrıca thread-dirmi? Xeyr. Əməliyyatın tamamlanmasını və nəticəsini təmsil edir.

Async CPU hesabını avtomatik sürətləndirirmi? Xeyr. Gözləməni bloklamadan idarə etməyə imkan verir; CPU işi üçün alqoritm və hesablama resursları ayrıca məsələdir.

WhenAll-da biri xəta atsa digərləri dayanırmı? Avtomatik dayanmır. Onların ləğvi ayrıca idarə olunur.

Bir cümləlik məntiq: işi başladırıq, tamamlanmanı Task ilə təmsil edirik, lazım olan nəticəni await ilə gözləyirik və işin resurs, xəta, ləğv sərhədlərini ayrıca düzgün saxlayırıq.